

PyCore Microarchitecture Diagrams

A figure atlas for the research paper

PyCore systems notes

`docs/paper/systems/pycore_diagrams.tex`

October 2026

Abstract

PyCore is a research CPU whose instruction set is a CPython 3.14 bytecode subset. This note is a diagram atlas of the hart as built: the two-core system, the control states, the tagged register ring, the memory hierarchy, and every accelerator (integer, floating-point, string, and the collector), plus the container layouts those accelerators read. Each figure is drawn against the RTL names in `pycore/rtl` so it can be lifted into a paper with the caption edited and the geometry left alone. Numbers are the parameters in `pycore_defs.svh`. The call binder's internal phases stay in the sibling note `call_fsm.tex`.

Contents

1	How to use these figures	8
2	The machine	8
3	Tagged values	11
4	Register stack	11
5	Memory system	11
6	Accelerators	16
6.1	Floating point	21
6.2	Strings	21
6.3	Garbage collection	21
7	Containers, calls, and traps	21
7.1	Iterator payload	26
8	Figure index for the paper	26

List of Figures

- 1 Two-core system. PyCore is the CPython-bytecode hart. Excore is an RV32 multicycle hart that finishes recoverable traps in firmware. Instruction memories stay private: fetch uses L1I, and excore has its own firmware memory. The shared resource is the data heap. PyCore's data masters share one port into L1D. Excore's slot port attaches at L2 and never at L1D. PyCore is frozen in `S_TRAP_MARSHAL` / `S_TRAP_WAIT` while excore owns memory. `pycore_system` is the same hart with the mailbox tied off. . . 9

2	PyCore hart. The machine is multi-cycle: one instruction is in flight, so the register file needs no forwarding. The steady path is fetch, decode, execute, a thin memory stage, and writeback. Container operations, the string accelerator, and calls leave that path and retire from their own states. <code>S_MEM</code> serves the internal pointer opcodes. Image bytecode reaches the heap from the container FSM, calls, <code>STRACC</code> , and the collector. <code>CODC</code> and <code>GIC</code> are combinational result caches, not line caches. State transitions are Figure 3.	9
3	Control states in <code>pycore_core</code> . The top row is the steady path; the arrow above it is the return to fetch. Solid edges are completions, not the self-loops that wait on <code>ack</code> or <code>stall_o</code> . Any latched fatal trap forces <code>S_HALT</code> . <code>S_FETCH</code> holds until <code>pycore_fetch</code> presents a real instruction (<code>CACHE</code> skipped, <code>EXTENDED_ARG</code> folded inside the fetch unit). Recoverable traps leave the container FSM for <code>S_TRAP_MARSHAL</code> when <code>EXCORE_EN=1</code> . A collection that starts because an allocation does not fit enters <code>S_GC_ALLOC</code> directly. An empty free-run list continues into <code>S_GC_ENTER</code>	10
4	One instruction in flight. There is no EX/MEM or MEM/WB forwarding, no load-use interlock, and no branch flush of younger instructions. Taken branches redirect <code>pycore_fetch</code> on the next entry to <code>S_FETCH</code> . Decode reads <code>tos_r</code> combinationally, so the address it forms already includes the previous instruction's stack update. . . .	10
5	Architectural register entry. Every value the machine stores in the register file is 132 bits, <code>{tag[3:0], value[127:0]}</code> (<code>PYCORE_ENTRY_WIDTH</code>). The tag travels with the value, so a silicon register file does not need a second tag SRAM. Helpers <code>pycore_get_tag</code> , <code>pycore_get_val</code> , and <code>pycore_make_entry</code> are the only legal way to slice an entry.	11
6	The sixteen tags. All four tag bits are allocated, so a new kind does not widen the register file: heap objects share <code>OBJECT</code> and carry <code>ob_kind</code> in the header, and mutable containers share <code>MUT_COLLEC</code> with a kind in <code>value[127:124]</code> . Arithmetic on <code>OBJECT</code> , <code>ITER</code> , <code>MUT_COLLEC</code> , unbound <code>CONTROL</code> , or a reserved tag traps unless a dedicated opcode path owns that tag. <code>INT</code> is a 64-bit signed fast path; a result that does not fit raises <code>PY_TRAP_OVERFLOW</code> instead of becoming a big integer.	12
7	String representation. A string is a <code>SHORT_STR</code> exactly when it is kind 1 (Latin-1) and holds at most 15 characters. Every other string is a <code>LONG_STR</code> handle whose payload lives in the object heap. <code>len</code> , <code>hash</code> , and <code>truthiness</code> read the handle and do not touch memory. The canonical rule is mandatory in both directions: the accelerator must return a <code>SHORT_STR</code> for every qualifying result and must not allocate a short <code>LONG_STR</code>	12
8	Operand ring, the common case <code>tos_r > rf_wm_r</code> . The physical file is 256 entries and does not shift. <code>tos_r</code> is the next free slot; the top value sits at <code>tos_r-1</code> . <code>locals_base</code> is the callee's slot 0. Frame 0 is an ordinary frame: nothing reserves <code>RF[0..31]</code> . The three pointers live in <code>pycore_core</code> . The regfile module's own push, pop, and TOS ports are tied off.	12
9	Spill and fill around a call. A callee must keep its whole window resident. The window is capped at <code>RF_WINDOW_CAP = 240</code> (256 entries minus a 16-entry reserve); a larger frame is <code>CALL_FILTER</code> . If the live suffix plus that window would exceed 256, <code>S_RF_SPILL</code> evicts the overflow plus <code>RF_SPILL_HYST = 32</code> entries, clamped to the resident count. Each entry is two 16-byte beats, value then tag, at the spill stack pointer. <code>S_RF_FILL</code> restores them on return until <code>locals_base</code> is inside the suffix. Spilling in the middle of a frame is not implemented.	12
10	Three different stacks. The call frame in <code>pycore_frame</code> stores pointers, not register values: return PC, TOS base, locals base, globals base, the saved instance, the return-mode bit, and the caller's code-object address. The operand values stay in the ring. Values that do not fit the ring move to the spill LIFO at <code>0xF40000</code> (256 KB). Return pops slot 1 then slot 0, restores those pointers, and reloads the caller's <code>co_consts</code> and <code>co_names</code>	13

- 11 How decode addresses the ring. `LOAD_FAST i` reads `locals_base+i`. Stack opcodes read and write relative to `tos_r`. `S_WB` commits one entry and applies the stack delta. Container, return, and spill-fill writeback share that single write port, in that priority. While `S_CONTAINER` runs, `rs1` is redirected to `container_rf_addr_r` so a multi-element build does not need a third read port. Unbound locals are `CONTROL/UNINIT`; reading one traps rather than returning leftover bits. 13
- 12 Memory hierarchy behind an unchanged port. A request is captured on the cycle `req` is high. `ack` pulses for one cycle when the data is ready, any latency later, with `fault` beside it. At most one ordinary request is outstanding per master. `PYCORE_CACHE_EN=0` is a combinational bypass to the next level. Line size is 64 B everywhere. L1 hit latency is 1 cycle; L2 hit latency is 8 cycles (`PYCORE_L2_HIT_CYCLES`), a model parameter. The crossbar adds `PYCORE_CODE_ADDR_BASE = 0x01000000` to instruction addresses so code and data do not alias in L2. CODC and GIC sit beside the line caches. The frame-top buffer (`PYCORE_FTB_FRAMES = 4`) is a named parameter only. 13
- 13 Address map. The heap is a bump pointer from `PYCORE_HEAP_BASE` with no free list until the collector is enabled. The static boot image occupies about 380 KB of that heap. Code is a separate slot space: the architectural PC is a slot index, and fetch sends `pc << 3`. `pycore_code_mem` is compiled and not instantiated; the live path is `pycore_mem_hier` into `pycore_ram`. 14
- 14 One `pycore_cache`, three instantiations (L1I, L1D, L2). L1I is read-only and write-invalidate / no-allocate: a store drops a hit line and forwards the write, and does not allocate. L1D and L2 are write-back and write-allocate. An all-zero full-line write on L1D bypasses allocation; STRACC uses full-line installs so a new result line is not filled only to be overwritten. `rdata_line` returns the whole responding line with `ack`, which is how the fetch buffer fills. 14
- 15 The two opt-in ports. Nothing changes for a master that ignores them: an ordinary request takes the same number of cycles as before. The collector is the first user. While marking, it prefetches the line of each object it pushes and the next lines of the range it is scanning. A prefetch fills L1D and does not produce an answer. 14
- 16 The only sharing event. There is no cache coherence protocol. PyCore issues no data request while excore owns memory, and excore polls the mailbox while PyCore owns it. L1I is not flushed on the handoff. L2 is left in place; excore's writes land there and become visible to PyCore after the L1D invalidate. A code-RAM blit or patch invalidates L1I and flushes CODC. `STORE_NAME / STORE_GLOBAL` and a change of `globals_base_r` flush the GIC. A return that restores the same globals base does not. 15
- 17 Result caches. Both answer in the same cycle as the lookup. Fill and LRU update on the following clock edge. `CACHE_EN=0` forces a miss. CODC is flushed on `MAKE_FUNCTION`, code release, `trap_res`, and reset. The GIC is flushed as a whole on purpose: consulting a dict version to do better would itself be a data-memory access. 15
- 18 Fetch. The image keeps CPython's `CACHE` and `EXTENDED_ARG` bytes. The slot index equals the CPython code-unit index, so branch arguments are not rewritten. The live code image is ROM plus RAM in the unified RAM, selected by the slot address. . . 15
- 19 Who drives the data port. The mux is a priority encode, but the owners are mutually exclusive by core state. PyCore never arbitrates two live masters in one cycle. The port is 128-bit data, 16-byte aligned, with a 16-bit write strobe. A misaligned or out-of-range access raises `ADDR_ALIGN` or `MEM_FAULT`. GC also has the non-blocking prefetch port into L1D, which is not this mux. 16

20	Accelerators, together. Three kinds of hardware sit off the simple pipeline. The execute fabric is tag-routed arithmetic and stays inside <code>S_EXEC</code> . The container FSM, the string accelerator, and the collector are multi-cycle masters of the data port; the core is frozen in their state until they finish. Excore is the firmware accelerator for the cases those units refuse to do in place: growing a container and the uncontaminated bulk updates. Negative integer powers never reach the integer power unit; execute promotes both operands and reissues them on the FPU.	17
21	Execute fabric (<code>pycore_exec</code>). Tag decode chooses the unit, the promotion, and the result tag. Promotion is a combinational rewrite of the 64-bit lane. The integer ALU, BOOL operations, short-string compare and concatenation, and the FPU's compares and unary operations finish in the issue cycle. Multiplier, divider, integer power, and the FPU hold <code>S_EXEC</code> through <code>stall_o</code> . The result is written back as a fresh 132-bit entry. An integer result is sign-extended from 64 bits into the high half.	17
22	Shared handshake of every multi-cycle arithmetic unit. <code>start</code> is a level. The unit accepts it when idle and not already in the done cycle, so a requester that holds <code>start</code> and swaps operands the cycle after <code>done</code> gets back-to-back issue. <code>busy</code> is the structural-hazard bit a later scoreboard would use. Traps (<code>div_zero</code> , integer overflow, FPU exception) replace <code>done</code> and are reported with <code>stall</code> low. The core leaves <code>S_EXEC</code> by dropping <code>start</code> , which is what aborts a unit today.	17
23	Integer ALU (<code>pycore_int_alu</code>), combinational in the issue cycle. It covers <code>+</code> <code>-</code> <code>«</code> <code>»</code> <code>&</code> <code> </code> <code>^</code> <code>~</code> , unary plus and minus, and the integer compares. Beside the result it reports two flags. <code>overflow_flag</code> is set when add, subtract, unary minus, or a shift leaves the signed 64-bit range. <code>value_error</code> is a negative shift count. Execute turns those into <code>PY_TRAP_OVERFLOW</code> and <code>PY_TRAP_VALUE</code> . Compares reuse the subtractor and ignore the overflow flag. A shift of 64 or more is the sign fill for <code>»</code> , and either zero or overflow for <code>«</code> , matching CPython's long-shift rules on a 64-bit limb.	17
24	Signed 64×64 multiplier (<code>pycore_mul</code> , <code>STEP</code> = 16). Each cycle multiplies the full multiplicand by one chunk of the multiplier and folds it into a shift-right accumulator. The multiplier is treated as unsigned chunks with only the top chunk signed, so the running sum is the exact signed product with no final correction. Four steps plus the registered result are 5 cycles. The low 64 bits are the Python product when it fits; a high half that is not the sign extension of the low half is <code>PY_TRAP_OVERFLOW</code> . The same multiplier is the engine behind integer power. <code>STEP</code> is the area-versus-latency knob; the rest of the core does not depend on it.	18
25	Floor divider (<code>pycore_div</code> , <code>RL</code> = 2) on <code>pycore_udiv_seq</code> . Only $n = \text{bits}(a) - \text{bits}(b) + 1$ quotient bits are iterated, rounded up to a multiple of two. Latency is $5 + \lceil n/2 \rceil$: 5 cycles when $ a < b $, 37 for a full 64-bit quotient. The five fixed cycles are accept, leading-zero and alignment, core load, core result, and the floor fix-up. Division by zero is combinational in the accept cycle and does not start the unit. <code>INT64_MIN // -1</code> overflows. The same restoring core with <code>SQRT</code> = 1 is the floating-point square root. 18	18
26	Integer power (<code>pycore_ipow</code>). A non-negative exponent does $\text{bits}(e) - 1$ squarings and $\text{popcount}(e) - 1$ extra multiplies, each a full multiplier pass, plus four cycles of sequencing. A zero exponent finishes in the issue cycle. A negative exponent is a float in Python and is re-routed before this unit. The unit traps as soon as an intermediate product leaves the signed 64-bit range, so the long chains that complete are bases 0, 1, and -1.	18
27	FPU (<code>pycore_fpu</code>). One adder, one multiplier, one divider, and one power unit. The divider is also the square root. The significand multiplier is a single <code>pycore_umul_seq</code> of 75×53, step 14, muxed between <code>fp_mul</code> (53×53, zero-extended) and <code>fp_pow</code> . The sequencer never runs those two at once. Scalar add, subtract, multiply, and divide pass through. Remainder, floor division, general powers, and complex arithmetic are sequences of those datapaths in CPython's order. Compares, <code>not</code> , and unary plus and minus are combinational.	19

28	Binary64 add and subtract (<code>pycore_fp_add</code>). Three registered stages; the round and pack of the last stage are combinational, so <code>done</code> is four cycles after accept. Specials (NaN, infinity) can be decided in C0 and carried forward. Subnormals, signed zeros, and the guard/round/sticky scheme are in the shared packer <code>pycore_f64_pack_round</code> . Each stage already has its own valid bit so the unit can later accept a new operation while an old one drains.	19
29	Binary64 multiply (<code>pycore_fp_mul</code>). The 53×53 significand product is iterative. At the default step, <code>done</code> is the sixth cycle counting accept as the first. The parent owns the multiplier; this unit only presents operands and waits for the product.	19
30	Divider, remainder, and square root (<code>pycore_fp_divrem</code> on <code>pycore_udiv_seq</code>). Division iterates 56 quotient bits, two per cycle. Square root uses the same remainder register and the same three subtractors, with the divisor multiples replaced by the root-digit candidates $q(8Q + q)$ for $q \in \{1, 2, 3\}$. The final remainder is the sticky bit, so the root is correctly rounded. The sequencer sends $x^{0.5}$ here (35 cycles including the power special-case setup) instead of through the general power unit.	19
31	Binary64 power (<code>pycore_fp_pow</code>): $x^y = 2^{y \log_2 x}$ for positive finite x and finite y . Both transcendental steps are digit recurrences on one fixed-point datapath (two barrel shifters and two adders per cycle). The only multiplier is the shared 75×53 unit. Log2 is about 80 cycles, the multiply 5, and exp2 about 72, plus a few cycles of setup. A power-of-two base skips the log stage. The sequencer adds the CPython special cases around this unit, so a general <code>float ** float</code> is 171 cycles, 91 for a power-of-two base, and 19 when overflow or underflow is decided at the multiply. $x^{\pm 1}$, x^2 , and x^{-2} stay on multiply and divide instead.	20
32	Complex sequences, all on the scalar datapaths. A complex value is two binary64 lanes. Addition and subtraction are two adds. Multiplication is CPython's four-multiply product. Division is Smith's algorithm. When that algorithm would return <code>nan+nanj</code> and exactly one operand holds an infinity, two further adds rebuild the result from signs. Finite operands skip that recovery and keep the 128-cycle path. A negative base raised to a fractional real power is not a complex result here; it traps <code>FPU_EXCEPTION</code>	20
33	String accelerator (<code>pycore_str_accel</code>) as a master. It runs only while the core is in <code>S_STRACC</code> , so it cannot race L1D. The command carries up to three tagged operands and the current heap pointer. A result that fits in a <code>SHORT_STR</code> is assembled in the destination register and does not allocate. A result that needs a heap object either advances the bump pointer or returns <code>res_need_heap</code> with the heap untouched, so the core can collect and re-dispatch. Small kind-1 concatenations whose result is at most 15 bytes never leave the execute fabric.	21
34	STRACC control. PREP resolves a fast path (identity, empty operands, a case-map that the handle flags already answer) or selects an engine. The memory loop issues one 128-bit beat, waits for <code>ack</code> , then steps the engine. Kind 1, 2, and 4 move 16, 8, and 4 code units per beat. Engines are copy, compare, search, hash, character, replace, join, trim, classify, map, affix, expand, and split.	21
35	String equality, cheapest test first. Mixed <code>SHORT_STR</code> and <code>LONG_STR</code> compare unequal, because the canonical form says they cannot be the same text. Same-tag long strings with matching metadata issue <code>SA_CMP</code> , including ordering, not only <code>==</code> . Dict and set probes take tier 3 when the metadata matches. The hash in the handle is FNV-1a over the payload.	21
36	Long-string object in the heap. The header mirrors the handle, with the address field replaced by zero. Payload bytes are fixed-width code units, as in CPython's <code>PyUnicodeObject</code> , not UTF-8. Case mapping and the <code>is*</code> classifiers are a Latin-1 lookup table in the map and classify lanes. A code point above U+00FF raises a recoverable firmware trap. That is a performance boundary: the operation is still defined, it is just not done in the accelerator. Structural operations (length, index, slice, compare, search, hash, split, trim, pad, <code>ord</code>) are exact for all of Unicode. . . .	22

37	Where a collection sits in the core. Collections start at instruction boundaries. An allocation that does not fit aborts that instruction before the first architectural commit, the core collects, and the instruction is fetched again. <code>S_GC_ENTER</code> waits until any outstanding data beat from the abort has acknowledged, so a skip-clear cannot treat that read data as the first word of the static prune map. If the free-run list still has a run, <code>S_GC_ALLOC</code> satisfies the request without a collection.	22
38	Mark engine inside <code>pycore_gc</code> . The tracer walks memory-resident roots and the slots of each popped object, and emits child handles into the queue. The marker tests the first granule of an extent, sets every granule, and pushes kinds that have children. The bitmap is the on-chip array <code>bm_q</code> (<code>PYCORE_GC_BITMAP_WORDS</code> words, one bit per 16-byte granule). The 256-entry on-chip stack spills its oldest 32 entries to <code>0xFC0000</code> when full and refills 32 when empty. A pop scans at most 128 slots; the rest of the range is pushed first as a continuation, so a wide container holds at most 129 stack entries. A child that still does not fit is left unmarked and the range is written once to the rescan list. When the stack drains, that range is scanned again. The collection is abandoned only when the rescan list itself is full.	23
39	Sweep and the free-run list. The sweep clears the bitmap as it scans, so the next collection starts from a clean map. After reset, and after an abandoned collection, the idle engine clears the bitmap in the background, one word per cycle. Free runs are not a moving collector: a live object stays at its address. The first 1024 run headers stay on chip. Further headers go to <code>PYCORE_GC_RUN_TABLE</code> . Past that table they occupy the run's first granule, so the list has no fixed cap. Allocator traffic in <code>S_GC_ALLOC</code> reads a header and zero-fills the granted line.	23
40	Mark-stack entry kinds. An entry is 67 bits: kind, size, and address. <code>K_TUPLE</code> is also the continuation kind for 32-byte slots (tuples, list buffers, set tables, dict order). <code>K_DICTT</code> continues a dict hash table, whose slots are 64 bytes. <code>K_OBJ</code> covers the <code>OBJECT</code> header kinds: instance, type, bound method, builtin, bytearray, exception, cell, and function. Strings are traced for the object header; the payload bytes hold no pointers.	23
41	Sequence and dict layout, 16-byte slots. A list is a stable 32-byte object plus a relocatable element buffer. Growing the buffer does not move the handle. Each element is a value slot and a tag slot (32 bytes). An empty list allocates no buffer. A tuple stores its length in the handle, so it has no header. A dict is a stable 48-byte object, an insertion-order key sidecar, and a relocatable open-addressed table. Empty and deleted table slots are <code>UNINIT</code> and <code>TOMBSTONE</code> . New keys append to the sidecar and bump <code>version</code> ; a value overwrite does neither. Iteration of a dict follows the sidecar, never hash-slot order.	24
42	General heap object. The handle is <code>OBJECT</code> plus the object address. The kind is not in the tag: <code>LOAD_ATTR</code> and <code>CALL</code> read <code>ob_head</code> first. Kinds in use are instance (dict at field 0), type (dict, base, name), bound method, builtin, exception, cell, and function (code object plus a cell tuple). Lists, dicts, and sets are <code>MUT_COLLEC</code> , not <code>OBJECT</code> . Strings are <code>SHORT_STR</code> or <code>LONG_STR</code> . Range has its own tag.	24
43	Container microcode (<code>S_CONTAINER</code>). Decode sets <code>dec_is_container</code> and execute transfers here, skipping <code>S_MEM</code> and <code>S_WB</code> . The phase register is <code>container_phase_r</code> . Shared phases read or write a header, a value slot, and a tag slot, then <code>CP_DONE</code> returns to fetch. Dict and set operations insert a probe loop. Attribute operations insert a walk of the instance dict and the type MRO. The arms live in the <code>pycore_cont_*.svh</code> files included into the core. Fifty-six of the sixty-four <code>CONT_*</code> codes are in use. A recoverable failure sets <code>trap_marshall_pending_r</code> and <code>CP_DONE</code> leaves for the mailbox instead of fetch.	24

44	What stays on PyCore, and what is finished by excore. Hashing, equality, and the open-addressed probe stay on the hart. Capacity changes and the long memmoves go to firmware, which must complete the semantic effect rather than resize and retry. The trap is raised before any register-file, heap, or data-memory commit, which is what makes a later RETRY legal. No current handler returns RETRY ; they return COMPLETED . Contaminated containers (an OBJECT key or element) and tuple-sourced set updates stay on PyCore in pycore_cont_bulk	24
45	Call, as it sits among the other machines. The phase-by-phase binder is the sibling note call_fsm.tex . A user function is a code object, except a closure, which is an OBK_FUNCTION wrapping the code object and a cell tuple. CALL checks the callable, binds positional and keyword arguments against co_varnames , then enters the frame manager. Builtins with a hardware fast path (len , range , set , two-argument max , int and str conversion) dispatch inside this FSM. Other builtin calls raise the recoverable PY_TRAP_BUILTIN_CALL . Return pops the descriptor, writes the result into the caller's ring, and fills any spilled prefix the caller needs.	25
46	Exception-info stack. The arena is 4 KB at 0xF00000 , 32 bytes per node, depth 128. It is a data-memory LIFO owned by pycore_exc_stack , separate from both the operand ring and the call-frame stack. The native-method table and the StopIteration sidecar sit in the same arena. FOR_ITER exhaustion compares the raised type by identity with the sidecar latched at boot, and redirects to POP_ITER without a user handler. A user raise with no handler is still fatal PY_TRAP_RAISE	25
47	Trap classes. pycore_trap_recoverable in pycore_defs.svh is the split. With excore disabled, a recoverable code is fatal too. COMPLETED applies the popped and pushed entries and resumes at the next instruction. RETRY redirects fetch to the same PC. FATAL enters pycore_trap with the firmware's code. NEED_HEAP collects if the collector is enabled, and halts otherwise.	25
48	Boot record, read by S_BOOT when BOOT_EN=1 . The walker checks the tags, caches co_consts and co_names , latches the globals and builtins bases, and redirects fetch to the module entry slot. A serialized code object is eight tagged fields: entry slot, consts, names, packed metadata, defaults, varnames, keyword defaults, and the raw exception table. LOAD_CONST indexes co_consts with two data reads (value, then tag). LOAD_GLOBAL and LOAD_NAME probe the frame globals and then the builtins dict; a miss is MEM_FAULT	25

1 How to use these figures

The atlas is the block diagram a reader needs before any opcode table. It is not a second architecture manual: `pycore/docs/architecture.md` remains the prose source of truth, and a figure that disagrees with that file or with the RTL is wrong.

Color. Blue is control state. Green is memory. Amber is the register file. Purple is an accelerator. Orange is excore or the mailbox. Red is a trap. A dashed box is specified and not built, or reserved.

Lifting a figure. Each figure is a separate file under `figures/`, and every picture uses the styles in `tikz_styles.tex`. Copy one `tikzpicture` into the paper, input the style file from the preamble, and rewrite the caption for the venue. The labels are RTL identifiers on purpose (`rf_wm_r`, `S_STRACC`, `CP_DICT_PROBE`).

What is in scope. The hart, its memory system, the accelerators, the container object model, and the excore contract. Out of scope are the on-device compiler, the firmware bodies under `pycore_firmware/`, and the excore pipeline itself. Those have their own documents.

Three facts that keep the diagrams honest. The core is multi-cycle and non-pipelined: one instruction is in flight. Integers are a signed 64-bit fast path; overflow traps instead of growing a big integer. The collector is built and off unless the simulation is started with `+GC_EN=1`.

2 The machine

PyCore’s ISA is the bytecode, not a RISC that a compiler lowers bytecode onto. That is why containers, calls, and strings are states in the same control machine as fetch, rather than coprocessor instructions issued by a host core.

Table 1 is the state register in `pycore_core.sv`. Figure 3 draws the edges that matter for a paper; it omits the self-loops where a state waits on `ack` or `stall_o`.

Table 1: Control states.

Id	State	Role
0	<code>S_FETCH</code>	Present one real instruction.
1	<code>S_DECODE</code>	Form ring addresses and latch operands.
2	<code>S_EXEC</code>	Execute fabric and branch unit.
3	<code>S_MEM</code>	Pointer load/store only.
4	<code>S_WB</code>	One register write, stack-pointer update.
5	<code>S_HALT</code>	Fatal trap. Architectural state freezes.
6	<code>S_CALL</code>	Bind arguments and push a frame.
7	<code>S_RETURN</code>	Pop a frame and write the result.
8	<code>S_CONTAINER</code>	Heap and name operations.
9	<code>S_BOOT</code>	Read the boot record.
10	<code>S_TRAP_MARSHAL</code>	Hand operands to the mailbox.
11	<code>S_TRAP_WAIT</code>	Apply <code>COMPLETED</code> / <code>RETRY</code> / <code>FATAL</code> .
12	<code>S_STRACC</code>	String accelerator owns the data port.
13	<code>S_RF_SPILL</code>	Evict a ring prefix to the spill LIFO.
14	<code>S_RF_FILL</code>	Restore a spilled prefix.
15	<code>S_RF_INIT</code>	Clear unfilled locals to <code>UNINIT</code> , 32 at a time.
16	<code>S_CODE_WRITE</code>	Core writes code RAM (blit / patch).
17	<code>S_GC_ENTER</code>	Drain an outstanding data beat, start the engine.
18	<code>S_GC_ROOTS</code>	Stream register and ring roots.
19	<code>S_GC_RUN</code>	Mark and sweep.
20	<code>S_GC_ALLOC</code>	Grant a free run, or collect if the list is empty.

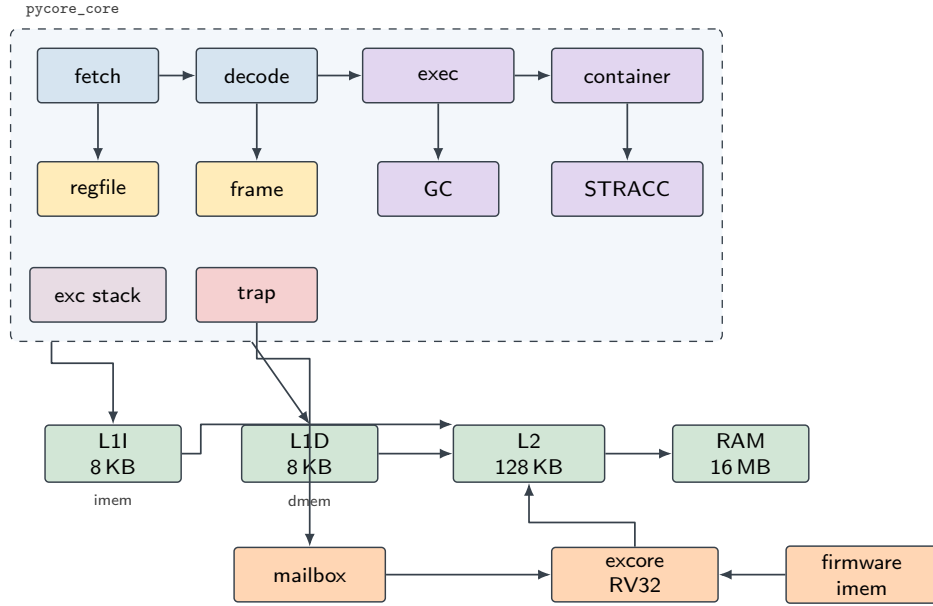


Figure 1: Two-core system. PyCore is the CPython-bytecode hart. Excore is an RV32 multicyle hart that finishes recoverable traps in firmware. Instruction memories stay private: fetch uses L1I, and excore has its own firmware memory. The shared resource is the data heap. PyCore’s data masters share one port into L1D. Excore’s slot port attaches at L2 and never at L1D. PyCore is frozen in `S_TRAP_MARSHAL` / `S_TRAP_WAIT` while excore owns memory. `pycore_system` is the same hart with the mailbox tied off.

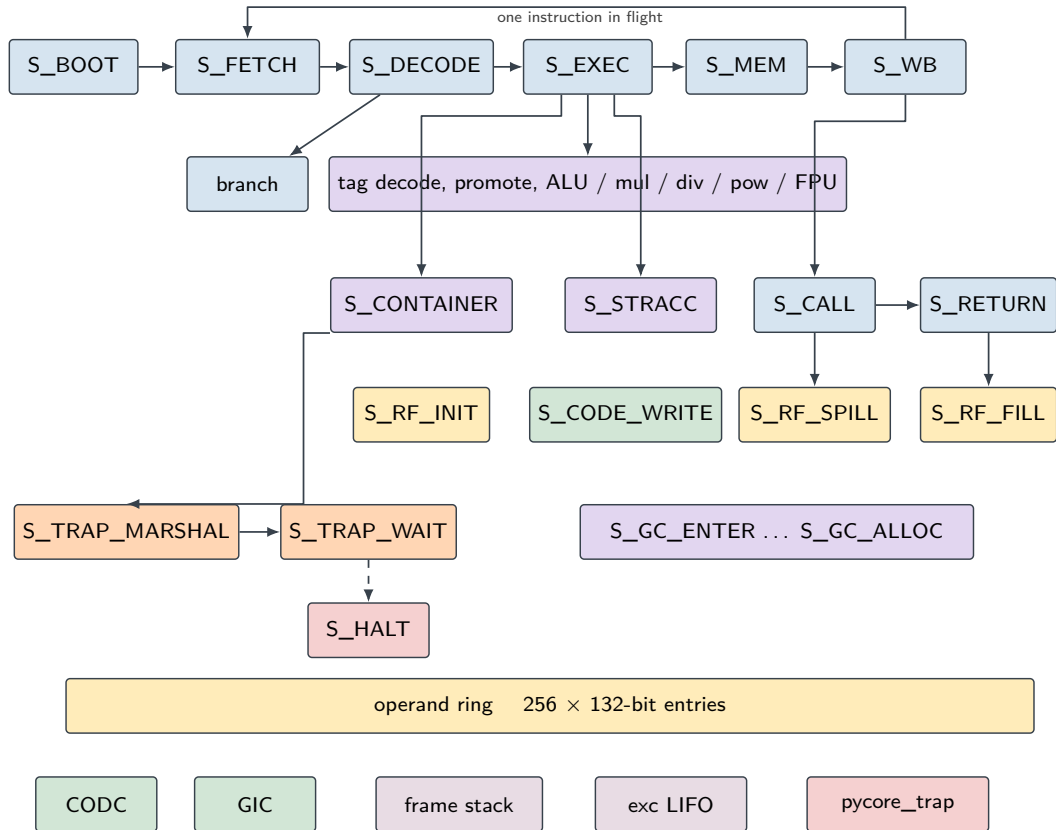


Figure 2: PyCore hart. The machine is multi-cycle: one instruction is in flight, so the register file needs no forwarding. The steady path is fetch, decode, execute, a thin memory stage, and writeback. Container operations, the string accelerator, and calls leave that path and retire from their own states. `S_MEM` serves the internal pointer opcodes. Image bytecode reaches the heap from the container FSM, calls, STRACC, and the collector. CODC and GIC are combinational result caches, not line caches. State transitions are Figure 3.

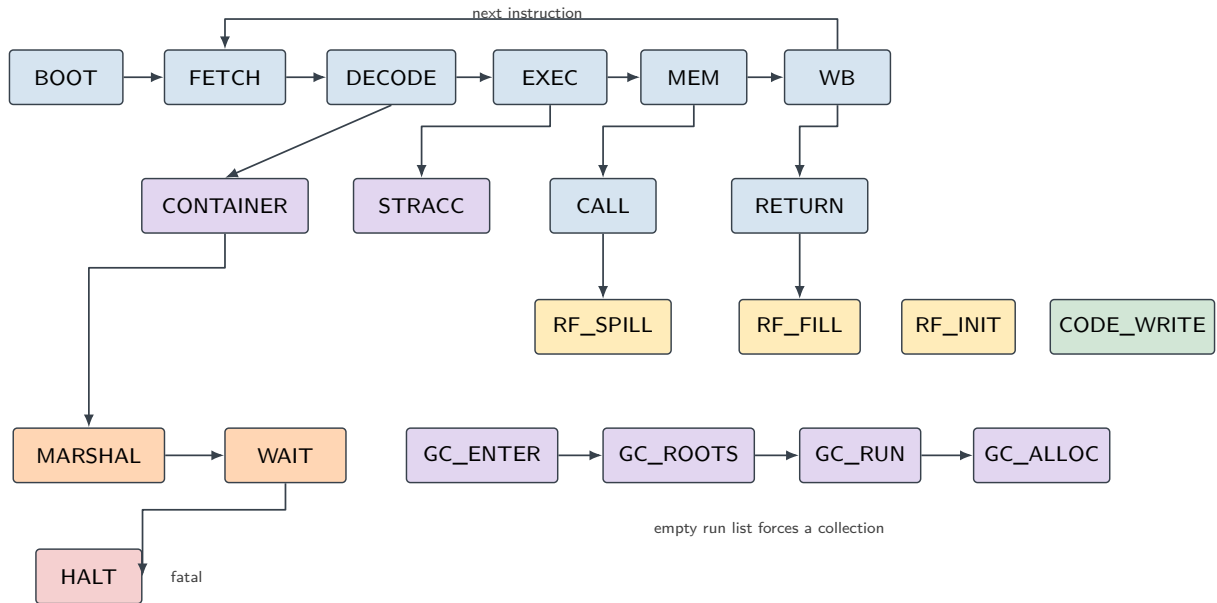


Figure 3: Control states in `pycore_core`. The top row is the steady path; the arrow above it is the return to fetch. Solid edges are completions, not the self-loops that wait on `ack` or `stall_o`. Any latched fatal trap forces `S_HALT`. `S_FETCH` holds until `pycore_fetch` presents a real instruction (`CACHE` skipped, `EXTENDED_ARG` folded inside the fetch unit). Recoverable traps leave the container FSM for `S_TRAP_MARSHAL` when `EXCORE_EN=1`. A collection that starts because an allocation does not fit enters `S_GC_ALLOC` directly. An empty free-run list continues into `S_GC_ENTER`.

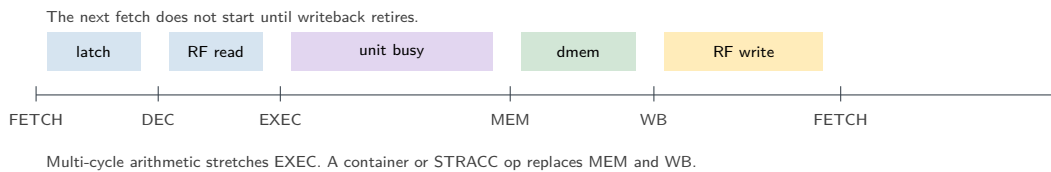


Figure 4: One instruction in flight. There is no EX/MEM or MEM/WB forwarding, no load-use interlock, and no branch flush of younger instructions. Taken branches redirect `pycore_fetch` on the next entry to `S_FETCH`. Decode reads `tos_r` combinationaly, so the address it forms already includes the previous instruction's stack update.

3 Tagged values

Every architectural value is one 132-bit entry. The tag is stored beside the payload so that a later custom register-file macro can replace `pycore_regfile` without a tag-lookup pipeline.

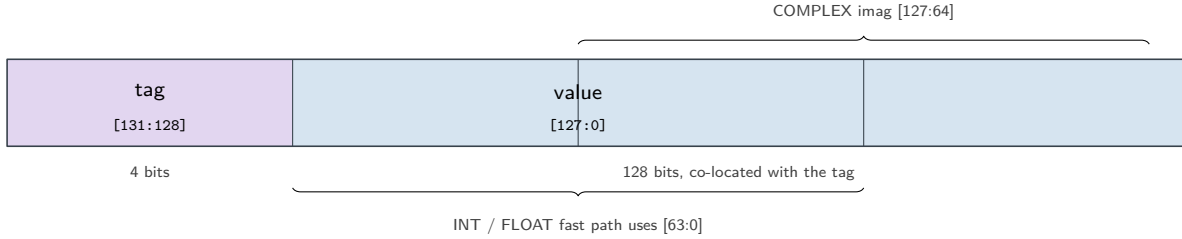


Figure 5: Architectural register entry. Every value the machine stores in the register file is 132 bits, `{tag[3:0], value[127:0]}` (`PYCORE_ENTRY_WIDTH`). The tag travels with the value, so a silicon register file does not need a second tag SRAM. Helpers `pycore_get_tag`, `pycore_get_val`, and `pycore_make_entry` are the only legal way to slice an entry.

4 Register stack

The operand stack and the locals of every live frame share one 256-entry ring. Pointers move; entries do not slide. A call that does not fit spills a prefix of the ring to data memory and restores it on return. The call-frame stack is a third structure and holds only descriptors.

Table 2: Ring and stack parameters (`pycore_defs.svh`).

Parameter	Value	Meaning
<code>PYCORE_RF_DEPTH</code>	256	Entries in the ring.
<code>PYCORE_RF_RESERVE</code>	16	Entries a callee window may not consume.
<code>PYCORE_RF_WINDOW_CAP</code>	240	Maximum resident window. Larger is <code>CALL_FILTER</code> .
<code>PYCORE_RF_SPILL_HYST</code>	32	Extra entries spilled so the next call does not spill immediately.
<code>PYCORE_RF_INIT_CHUNK</code>	32	Unfilled locals cleared per <code>S_RF_INIT</code> burst.
<code>PYCORE_RF_SPILL_BASE</code>	0xF40000	Spill LIFO, 256 KB, 8192 entries.
<code>PYCORE_FRAME_STACK_BASE</code>	0xF01000	Frame descriptors, 32 KB, 1024 frames.

The register file has two combinational read ports and one write port. `S_WB`, container writeback, return writeback, and spill-fill writeback share the write port. Container operations may redirect the first read address for the duration of `S_CONTAINER`. There is no forwarding network because the previous instruction has retired before decode of the next one reads the file.

5 Memory system

The hart presents two master ports, instruction and data. Behind them is a modified Harvard hierarchy: split level-1 caches, a unified inclusive level-2, and a 16 MB RAM. The port contract is the one the core was written against when memory was a single-cycle SRAM, so every cache drops in underneath without retiming the control machine.

Data width on the CPU side of L1D is 128 bits with a 16-bit strobe. Instruction width is 64 bits, one slot. A 64-byte line is four data beats.

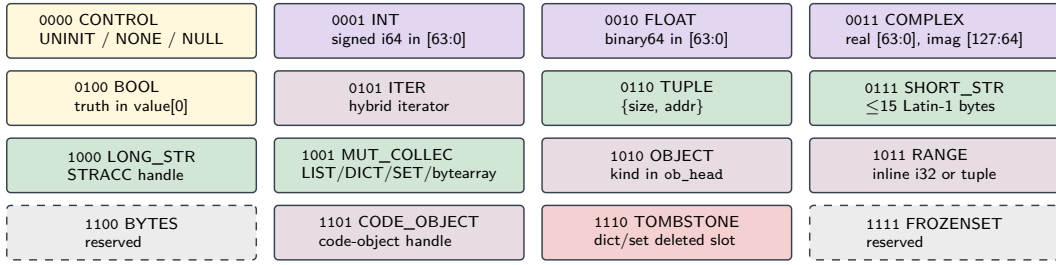


Figure 6: The sixteen tags. All four tag bits are allocated, so a new kind does not widen the register file: heap objects share **OBJECT** and carry **ob_kind** in the header, and mutable containers share **MUT_COLLEC** with a kind in **value[127:124]**. Arithmetic on **OBJECT**, **ITER**, **MUT_COLLEC**, unbound **CONTROL**, or a reserved tag traps unless a dedicated opcode path owns that tag. **INT** is a 64-bit signed fast path; a result that does not fit raises **PY_TRAP_OVERFLOW** instead of becoming a big integer.

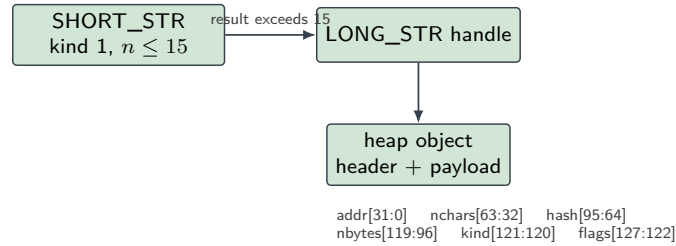


Figure 7: String representation. A string is a **SHORT_STR** exactly when it is kind 1 (Latin-1) and holds at most 15 characters. Every other string is a **LONG_STR** handle whose payload lives in the object heap. **len**, **hash**, and **truthiness** read the handle and do not touch memory. The canonical rule is mandatory in both directions: the accelerator must return a **SHORT_STR** for every qualifying result and must not allocate a short **LONG_STR**.

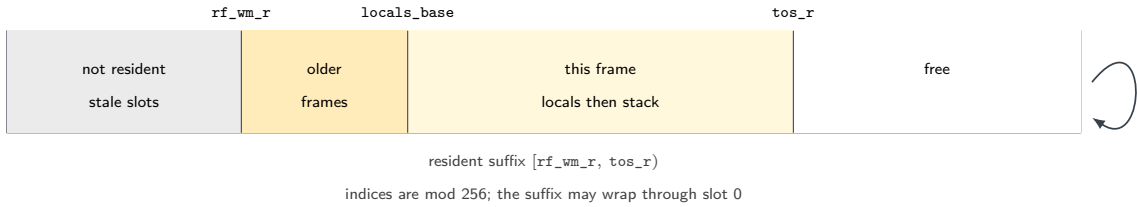


Figure 8: Operand ring, the common case $tos_r > rf_wm_r$. The physical file is 256 entries and does not shift. tos_r is the next free slot; the top value sits at $tos_r - 1$. $locals_base$ is the callee's slot 0. Frame 0 is an ordinary frame: nothing reserves **RF**[0..31]. The three pointers live in **pycore_core**. The regfile module's own push, pop, and TOS ports are tied off.

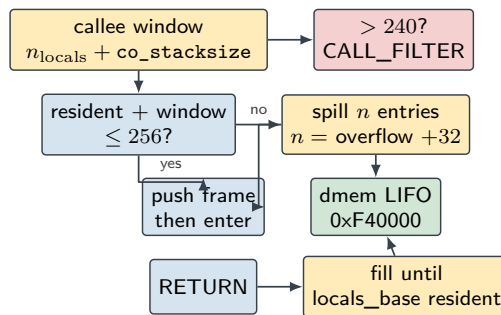


Figure 9: Spill and fill around a call. A callee must keep its whole window resident. The window is capped at **RF_WINDOW_CAP** = 240 (256 entries minus a 16-entry reserve); a larger frame is **CALL_FILTER**. If the live suffix plus that window would exceed 256, **S_RF_SPILL** evicts the overflow plus **RF_SPILL_HYST** = 32 entries, clamped to the resident count. Each entry is two 16-byte beats, value then tag, at the spill stack pointer. **S_RF_FILL** restores them on return until **locals_base** is inside the suffix. Spilling in the middle of a frame is not implemented.

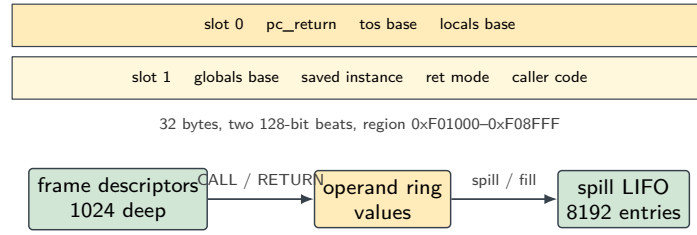


Figure 10: Three different stacks. The call frame in `pycore_frame` stores pointers, not register values: return PC, TOS base, locals base, globals base, the saved instance, the return-mode bit, and the caller’s code-object address. The operand values stay in the ring. Values that do not fit the ring move to the spill LIFO at 0xF40000 (256 KB). Return pops slot 1 then slot 0, restores those pointers, and reloads the caller’s `co_consts` and `co_names`.

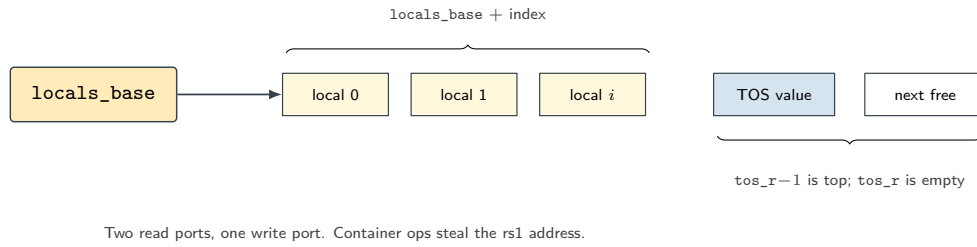


Figure 11: How decode addresses the ring. `LOAD_FAST i` reads `locals_base + i`. Stack opcodes read and write relative to `tos_r`. `S_WB` commits one entry and applies the stack delta. Container, return, and spill-fill writeback share that single write port, in that priority. While `S_CONTAINER` runs, `rs1` is redirected to `container_rf_addr_r` so a multi-element build does not need a third read port. Unbound locals are `CONTROL/UNINIT`; reading one traps rather than returning leftover bits.

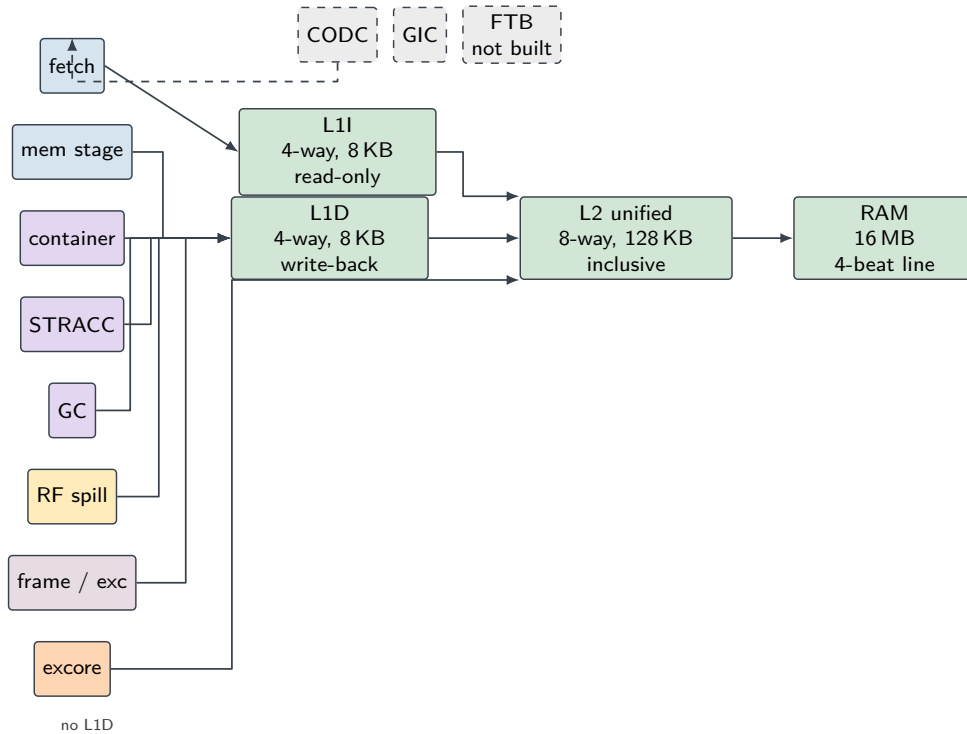


Figure 12: Memory hierarchy behind an unchanged port. A request is captured on the cycle `req` is high. `ack` pulses for one cycle when the data is ready, any latency later, with `fault` beside it. At most one ordinary request is outstanding per master. `PYCORE_CACHE_EN=0` is a combinational bypass to the next level. Line size is 64 B everywhere. L1 hit latency is 1 cycle; L2 hit latency is 8 cycles (`PYCORE_L2_HIT_CYCLES`), a model parameter. The crossbar adds `PYCORE_CODE_ADDR_BASE = 0x01000000` to instruction addresses so code and data do not alias in L2. CODC and GIC sit beside the line caches. The frame-top buffer (`PYCORE_FTB_FRAMES = 4`) is a named parameter only.

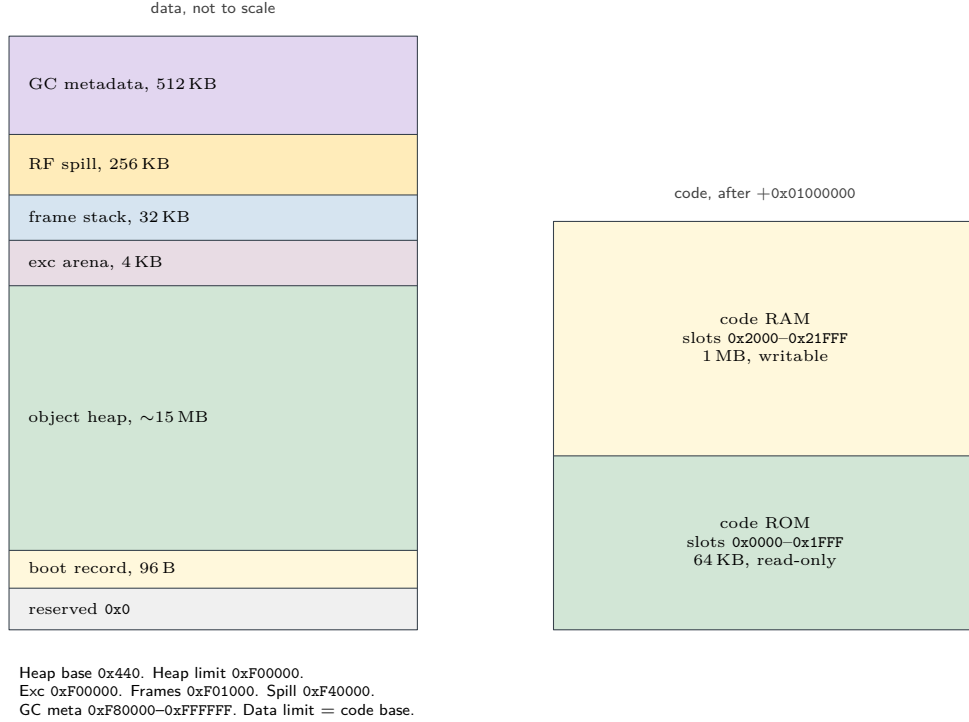


Figure 13: Address map. The heap is a bump pointer from PYCORE_HEAP_BASE with no free list until the collector is enabled. The static boot image occupies about 380 KB of that heap. Code is a separate slot space: the architectural PC is a slot index, and fetch sends $pc \ll 3$. `pycore_code_mem` is compiled and not instantiated; the live path is `pycore_mem_hier` into `pycore_ram`.

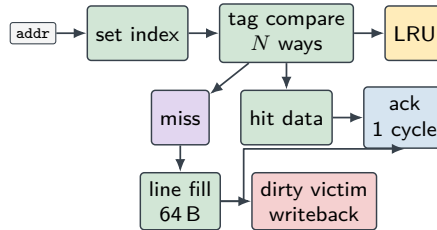


Figure 14: One `pycore_cache`, three instantiations (L1I, L1D, L2). L1I is read-only and write-invalidate / no-allocate: a store drops a hit line and forwards the write, and does not allocate. L1D and L2 are write-back and write-allocate. An all-zero full-line write on L1D bypasses allocation; STRACC uses full-line installs so a new result line is not filled only to be overwritten. `rdata_line` returns the whole responding line with `ack`, which is how the fetch buffer fills.

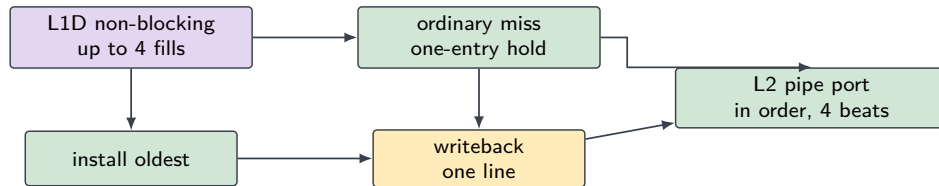


Figure 15: The two opt-in ports. Nothing changes for a master that ignores them: an ordinary request takes the same number of cycles as before. The collector is the first user. While marking, it prefetches the line of each object it pushes and the next lines of the range it is scanning. A prefetch fills L1D and does not produce an answer.

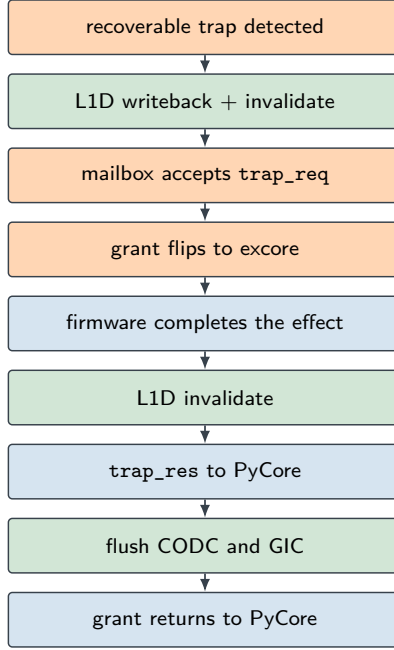


Figure 16: The only sharing event. There is no cache coherence protocol. PyCore issues no data request while excore owns memory, and excore polls the mailbox while PyCore owns it. L1I is not flushed on the handoff. L2 is left in place; excore’s writes land there and become visible to PyCore after the L1D invalidate. A code-RAM blit or patch invalidates L1I and flushes CODC. `STORE_NAME` / `STORE_GLOBAL` and a change of `globals_base_r` flush the GIC. A return that restores the same globals base does not.

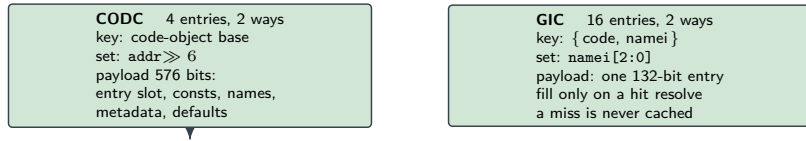


Figure 17: Result caches. Both answer in the same cycle as the lookup. Fill and LRU update on the following clock edge. `CACHE_EN=0` forces a miss. CODC is flushed on `MAKE_FUNCTION`, code release, `trap_res`, and reset. The GIC is flushed as a whole on purpose: consulting a dict version to do better would itself be a data-memory access.



Later slots in the same line, including the skips, do not issue another imem request.
L1I hit and miss counters therefore count fills behind this buffer, not wordcode slots.
A code-RAM write drops the buffer so a later fetch cannot observe a stale slot.

Figure 18: Fetch. The image keeps CPython’s `CACHE` and `EXTENDED_ARG` bytes. The slot index equals the CPython code-unit index, so branch arguments are not rewritten. The live code image is ROM plus RAM in the unified RAM, selected by the slot address.

Table 3: Line caches. Hit latencies are model parameters.

Level	Size	Ways	Line	Hit	Policy
L1I	8 KB	4	64 B	1	read-only, write-invalidate
L1D	8 KB	4	64 B	1	write-back, write-allocate, LRU
L2	128 KB	8	64 B	8	unified, inclusive, write-back
RAM	16 MB	—	64 B	—	4 beats; CI latency 4

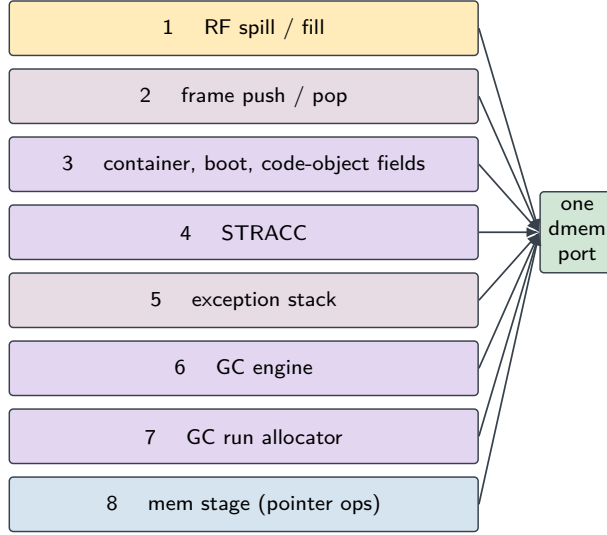


Figure 19: Who drives the data port. The mux is a priority encode, but the owners are mutually exclusive by core state. PyCore never arbitrates two live masters in one cycle. The port is 128-bit data, 16-byte aligned, with a 16-bit write strobe. A misaligned or out-of-range access raises `ADDR_ALIGN` or `MEM_FAULT`. GC also has the non-blocking prefetch port into L1D, which is not this mux.

6 Accelerators

An accelerator here is any unit that gives a bytecode operation a datapath shorter than a firmware trap. They fall into three places, drawn together in Figure 20 and then one at a time.

1. The execute fabric, inside `S_EXEC`: integer ALU, multiplier, divider, integer power, and the FPU.
2. Port-owning engines that freeze the core: the string accelerator and the collector, plus the container FSM, which is microcoded in the core rather than a separate module.
3. Excore, for the grow and bulk-update cases the container FSM raises before it has committed anything.

Table 4: Arithmetic latency, in cycles, counting the accept cycle. Divider latency depends on the number of quotient bits n . Integer power does $k = \text{bits}(e) - 1$ squarings and $p = \text{popcount}(e) - 1$ extra multiplies.

Unit	Operation	Cycles
<code>pycore_int_alu</code>	add, logic, shift, compare	1
<code>pycore_mul</code>	64×64 signed, step 16	5
<code>pycore_div</code>	floor division and remainder	$5 + \lceil n/2 \rceil$
<code>pycore_ipow</code>	non-negative integer power	$4 + 6k + 5p$
<code>pycore_fp_add</code>	binary64 add / sub	4
<code>pycore_fp_mul</code>	binary64 multiply	6
<code>pycore_fp_divrem</code>	binary64 divide or square root	32
<code>pycore_fp_divrem</code>	<code>fmod</code> , exponent gap $d \geq 0$	$5 + \lceil (d+1)/2 \rceil$
<code>pycore_fp_pow</code>	general binary64 power	166
<code>pycore_fpu</code>	general float <code>**</code> float	171
<code>pycore_fpu</code>	complex add / mul / div	10 / 34 / 128

`NB_TRUE_DIVIDE` always yields `FLOAT`, including `INT / INT`: both operands are converted to binary64 before the divider. `INT ** INT` with a negative exponent takes that same floating-point path. A negative base with a fractional exponent traps; the FPU has no complex power.

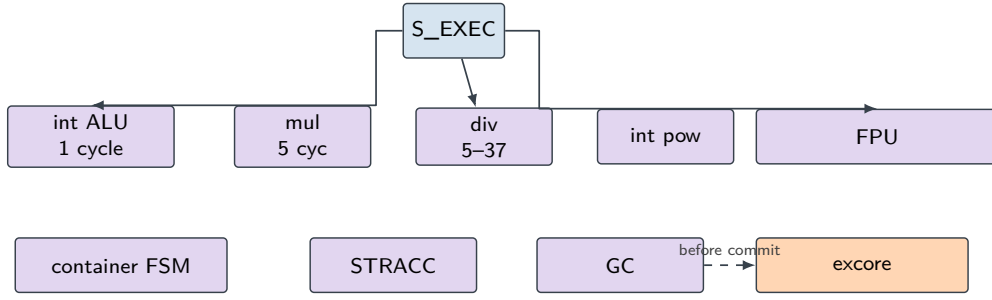


Figure 20: Accelerators, together. Three kinds of hardware sit off the simple pipeline. The execute fabric is tag-routed arithmetic and stays inside `S_EXEC`. The container FSM, the string accelerator, and the collector are multi-cycle masters of the data port; the core is frozen in their state until they finish. Excore is the firmware accelerator for the cases those units refuse to do in place: growing a container and the uncontaminated bulk updates. Negative integer powers never reach the integer power unit; execute promotes both operands and reissues them on the FPU.

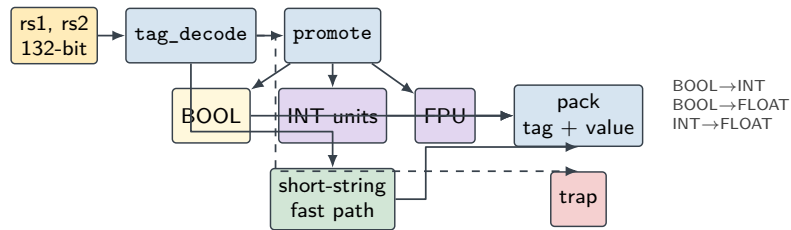


Figure 21: Execute fabric (`pycore_exec`). Tag decode chooses the unit, the promotion, and the result tag. Promotion is a combinational rewrite of the 64-bit lane. The integer ALU, BOOL operations, short-string compare and concatenation, and the FPU's compares and unary operations finish in the issue cycle. Multiplier, divider, integer power, and the FPU hold `S_EXEC` through `stall_o`. The result is written back as a fresh 132-bit entry. An integer result is sign-extended from 64 bits into the high half.

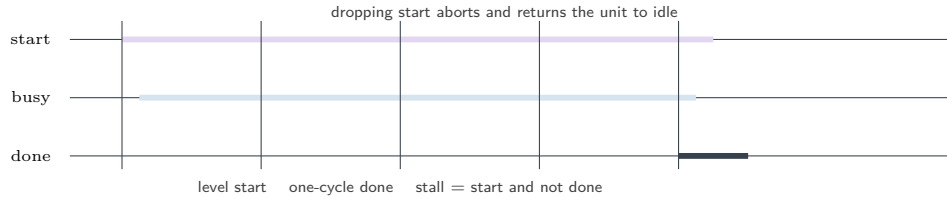


Figure 22: Shared handshake of every multi-cycle arithmetic unit. `start` is a level. The unit accepts it when idle and not already in the done cycle, so a requester that holds `start` and swaps operands the cycle after `done` gets back-to-back issue. `busy` is the structural-hazard bit a later scoreboard would use. Traps (`div_zero`, integer overflow, FPU exception) replace `done` and are reported with `stall` low. The core leaves `S_EXEC` by dropping `start`, which is what aborts a unit today.

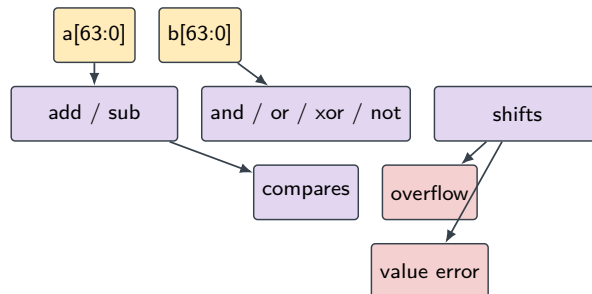


Figure 23: Integer ALU (`pycore_int_alu`), combinational in the issue cycle. It covers `+` `-` `«` `»` `&` `|` `^` `~`, unary plus and minus, and the integer compares. Beside the result it reports two flags. `overflow_flag` is set when add, subtract, unary minus, or a shift leaves the signed 64-bit range. `value_error` is a negative shift count. Execute turns those into `PY_TRAP_OVERFLOW` and `PY_TRAP_VALUE`. Compares reuse the subtractor and ignore the overflow flag. A shift of 64 or more is the sign fill for `»`, and either zero or overflow for `«`, matching CPython's long-shift rules on a 64-bit limb.

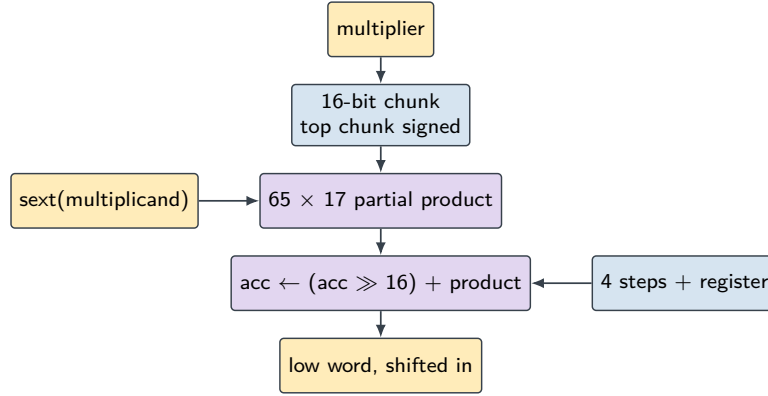


Figure 24: Signed 64×64 multiplier (`pycore_mul`, `STEP= 16`). Each cycle multiplies the full multiplicand by one chunk of the multiplier and folds it into a shift-right accumulator. The multiplier is treated as unsigned chunks with only the top chunk signed, so the running sum is the exact signed product with no final correction. Four steps plus the registered result are 5 cycles. The low 64 bits are the Python product when it fits; a high half that is not the sign extension of the low half is `PY_TRAP_OVERFLOW`. The same multiplier is the engine behind integer power. `STEP` is the area-versus-latency knob; the rest of the core does not depend on it.

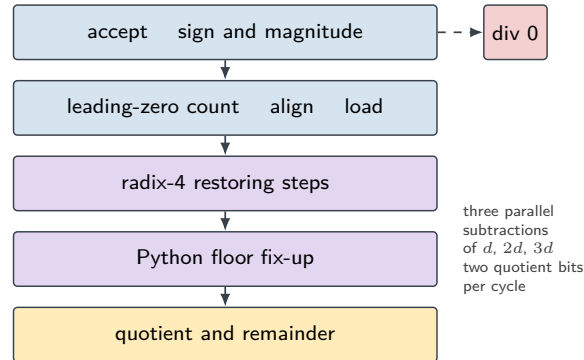


Figure 25: Floor divider (`pycore_div`, `RL= 2`) on `pycore_udiv_seq`. Only $n = \text{bits}(|a|) - \text{bits}(|b|) + 1$ quotient bits are iterated, rounded up to a multiple of two. Latency is $5 + \lceil n/2 \rceil$: 5 cycles when $|a| < |b|$, 37 for a full 64-bit quotient. The five fixed cycles are accept, leading-zero and alignment, core load, core result, and the floor fix-up. Division by zero is combinational in the accept cycle and does not start the unit. `INT64_MIN // -1` overflows. The same restoring core with `SQRT= 1` is the floating-point square root.

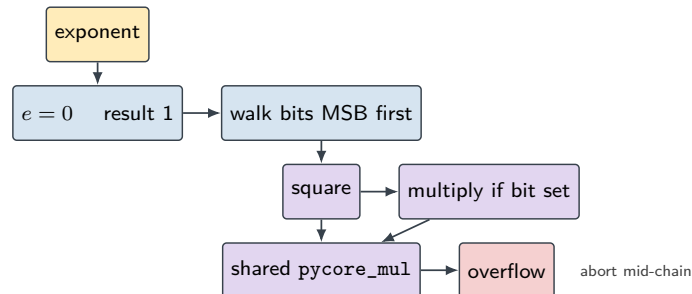


Figure 26: Integer power (`pycore_ipow`). A non-negative exponent does $\text{bits}(e) - 1$ squarings and $\text{popcount}(e) - 1$ extra multiplies, each a full multiplier pass, plus four cycles of sequencing. A zero exponent finishes in the issue cycle. A negative exponent is a float in Python and is re-routed before this unit. The unit traps as soon as an intermediate product leaves the signed 64-bit range, so the long chains that complete are bases 0, 1, and -1 .

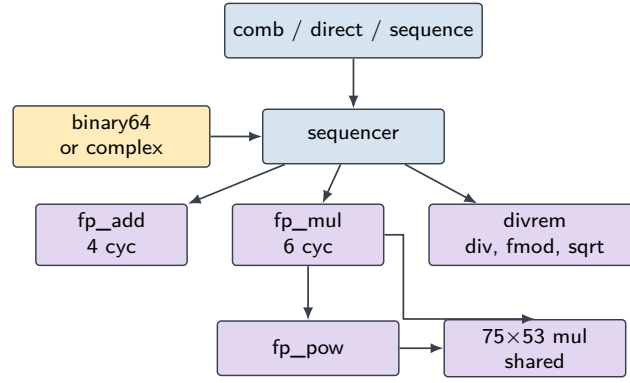


Figure 27: FPU (`pycore_fpu`). One adder, one multiplier, one divider, and one power unit. The divider is also the square root. The significand multiplier is a single `pycore_umul_seq` of 75×53 , step 14, muxed between `fp_mul` (53×53 , zero-extended) and `fp_pow`. The sequencer never runs those two at once. Scalar add, subtract, multiply, and divide pass through. Remainder, floor division, general powers, and complex arithmetic are sequences of those datapaths in CPython’s order. Compares, `not`, and unary plus and minus are combinational.

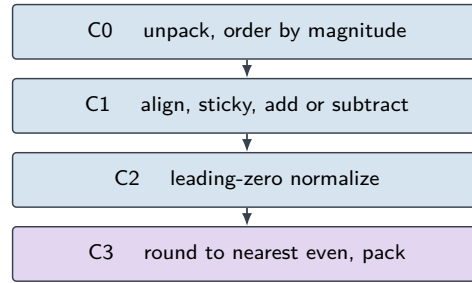


Figure 28: Binary64 add and subtract (`pycore_fp_add`). Three registered stages; the round and pack of the last stage are combinational, so `done` is four cycles after accept. Specials (NaN, infinity) can be decided in C0 and carried forward. Subnormals, signed zeros, and the guard/round/sticky scheme are in the shared packer `pycore_f64_pack_round`. Each stage already has its own valid bit so the unit can later accept a new operation while an old one drains.

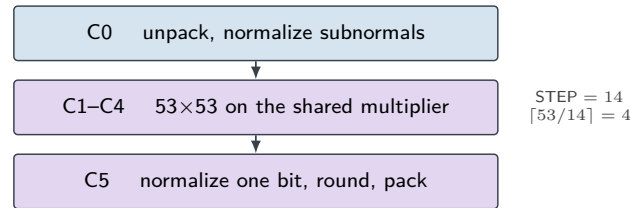


Figure 29: Binary64 multiply (`pycore_fp_mul`). The 53×53 significand product is iterative. At the default step, `done` is the sixth cycle counting accept as the first. The parent owns the multiplier; this unit only presents operands and waits for the product.

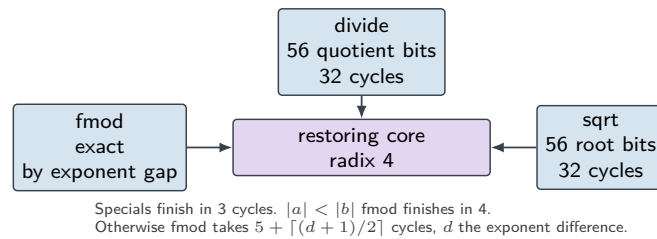


Figure 30: Divider, remainder, and square root (`pycore_fp_divrem` on `pycore_udiv_seq`). Division iterates 56 quotient bits, two per cycle. Square root uses the same remainder register and the same three subtractors, with the divisor multiples replaced by the root-digit candidates $q(8Q + q)$ for $q \in \{1, 2, 3\}$. The final remainder is the sticky bit, so the root is correctly rounded. The sequencer sends $x^{0.5}$ here (35 cycles including the power special-case setup) instead of through the general power unit.

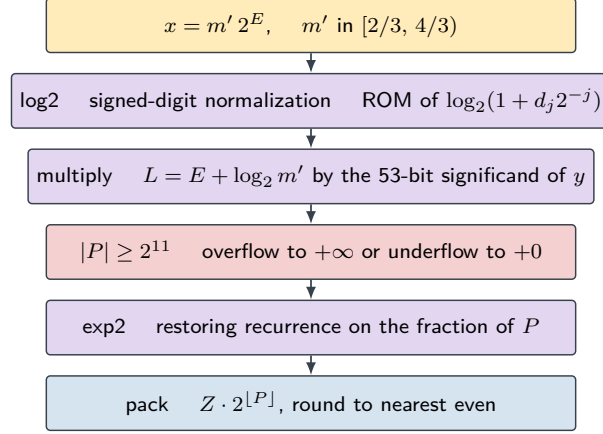


Figure 31: Binary64 power (`pycore_fp_pow`): $x^y = 2^{y \log_2 x}$ for positive finite x and finite y . Both transcendental steps are digit recurrences on one fixed-point datapath (two barrel shifters and two adders per cycle). The only multiplier is the shared 75×53 unit. Log2 is about 80 cycles, the multiply 5, and exp2 about 72, plus a few cycles of setup. A power-of-two base skips the log stage. The sequencer adds the CPython special cases around this unit, so a general `float ** float` is 171 cycles, 91 for a power-of-two base, and 19 when overflow or underflow is decided at the multiply. $x^{\pm 1}$, x^2 , and x^{-2} stay on multiply and divide instead.

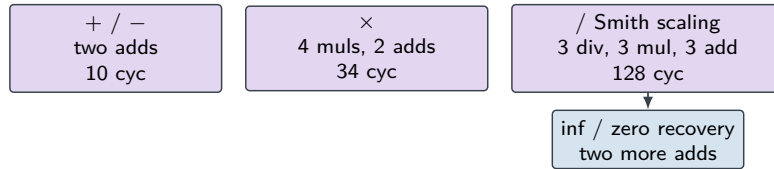


Figure 32: Complex sequences, all on the scalar datapaths. A complex value is two binary64 lanes. Addition and subtraction are two adds. Multiplication is CPython’s four-multiply product. Division is Smith’s algorithm. When that algorithm would return `nan+nanj` and exactly one operand holds an infinity, two further adds rebuild the result from signs. Finite operands skip that recovery and keep the 128-cycle path. A negative base raised to a fractional real power is not a complex result here; it traps `FPU_EXCEPTION`.

6.1 Floating point

6.2 Strings

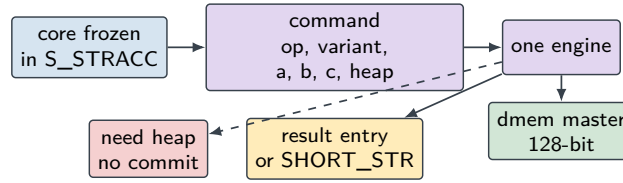


Figure 33: String accelerator (`pycore_str_accel`) as a master. It runs only while the core is in `S_STRACC`, so it cannot race L1D. The command carries up to three tagged operands and the current heap pointer. A result that fits in a `SHORT_STR` is assembled in the destination register and does not allocate. A result that needs a heap object either advances the bump pointer or returns `res_need_heap` with the heap untouched, so the core can collect and re-dispatch. Small kind-1 concatenations whose result is at most 15 bytes never leave the execute fabric.

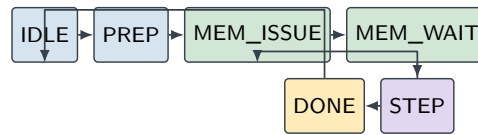
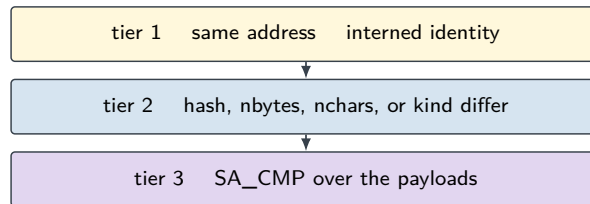


Figure 34: STRACC control. `PREP` resolves a fast path (identity, empty operands, a case-map that the handle flags already answer) or selects an engine. The memory loop issues one 128-bit beat, waits for `ack`, then steps the engine. Kind 1, 2, and 4 move 16, 8, and 4 code units per beat. Engines are copy, compare, search, hash, character, replace, join, trim, classify, map, affix, expand, and split.



Interning is an optimization. Runtime concatenations are valid dict keys.

Figure 35: String equality, cheapest test first. Mixed `SHORT_STR` and `LONG_STR` compare unequal, because the canonical form says they cannot be the same text. Same-tag long strings with matching metadata issue `SA_CMP`, including ordering, not only `==`. Dict and set probes take tier 3 when the metadata matches. The hash in the handle is FNV-1a over the payload.

6.3 Garbage collection

The collector is precise: it traces from tags, not from a conservative scan of every word. It does not move objects, so a handle cached in the ring or in a CODC payload stays valid across a collection. Roots are the live ring suffix, the frame descriptors, the exception stack, the spill LIFO, the boot record, and the extra roots the image records for builtins that the static prune map would otherwise hide.

7 Containers, calls, and traps

The container FSM is the accelerator for everything whose payload is a pointer graph rather than a scalar. Figure 41 is the memory image those phases walk. Figure 44 is the cut between what the hart finishes and what it hands to excore.

`BUILD_SET` and `SET_ADD` probe and insert on PyCore. A load of two thirds or more raises `SET_GROW`. `DELETE_SUBSCR` and `STORE_SUBSCR` on a set are type errors.

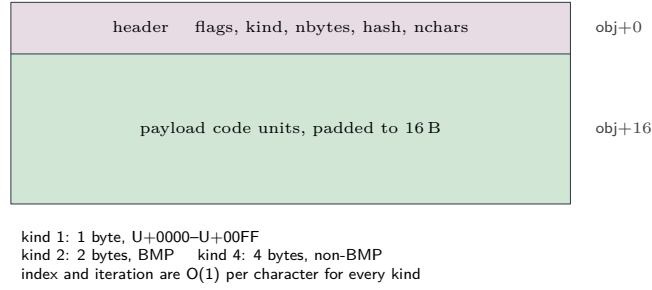


Figure 36: Long-string object in the heap. The header mirrors the handle, with the address field replaced by zero. Payload bytes are fixed-width code units, as in CPython’s `PyUnicodeObject`, not UTF-8. Case mapping and the `is*` classifiers are a Latin-1 lookup table in the map and classify lanes. A code point above U+00FF raises a recoverable firmware trap. That is a performance boundary: the operation is still defined, it is just not done in the accelerator. Structural operations (length, index, slice, compare, search, hash, split, trim, pad, ord) are exact for all of Unicode.

Table 5: STRACC commands (`PY_SA_*`). The variant field selects a direction, a character class, or a case map.

Id	Command	Use
0	CONCAT	Long concatenation. Short results stay on the ALU.
1	REPEAT	String repeat.
2	SLICE	Slice of a long string.
3	PAD	Left, right, or both.
4	CMP	Payload compare after the handle rejects fail.
5	SEARCH	Find, count, contains, prefix, suffix.
6	HASH	FNV-1a into the handle.
7	CHAR_AT	O(1) index, every kind.
8	ITER_NEXT	One character per <code>FOR_ITER</code> step.
9	REPLACE	Substring replace.
10	JOIN	Join over a list of strings.
11	TRIM	Left, right, or both.
12	CLASSIFY	Latin-1 <code>is*</code> tests.
13	MAP	Latin-1 case map.
14	SPLIT	Split, lines, partition.
15	AFFIX	Prefix or suffix strip.
16	ZFILL	Zero fill.
17	EXPANDTABS	Tab expansion.
18	ORD	One long character to an integer.
19	CHR	Integer code point to one character.

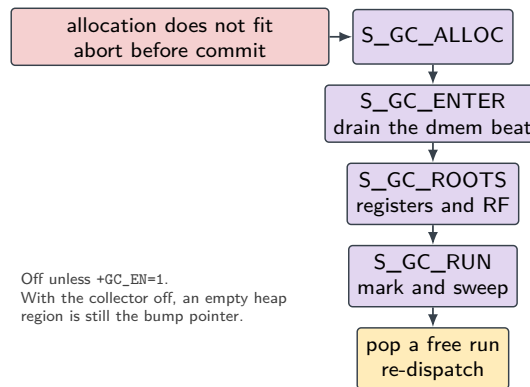
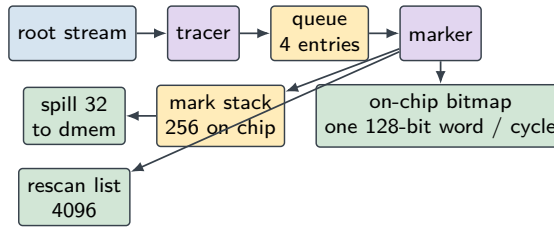


Figure 37: Where a collection sits in the core. Collections start at instruction boundaries. An allocation that does not fit aborts that instruction before the first architectural commit, the core collects, and the instruction is fetched again. `S_GC_ENTER` waits until any outstanding data beat from the abort has acknowledged, so a skip-clear cannot treat that read data as the first word of the static prune map. If the free-run list still has a run, `S_GC_ALLOC` satisfies the request without a collection.



One outstanding request. The marker wins a tie with the tracer.

Figure 38: Mark engine inside `pycore_gc`. The tracer walks memory-resident roots and the slots of each popped object, and emits child handles into the queue. The marker tests the first granule of an extent, sets every granule, and pushes kinds that have children. The bitmap is the on-chip array `bm_q` (`PYCORE_GC_BITMAP_WORDS` words, one bit per 16-byte granule). The 256-entry on-chip stack spills its oldest 32 entries to `0xFC0000` when full and refills 32 when empty. A pop scans at most 128 slots; the rest of the range is pushed first as a continuation, so a wide container holds at most 129 stack entries. A child that still does not fit is left unmarked and the range is written once to the rescan list. When the stack drains, that range is scanned again. The collection is abandoned only when the rescan list itself is full.

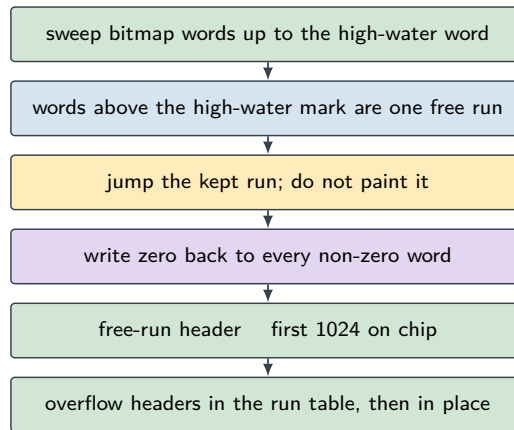


Figure 39: Sweep and the free-run list. The sweep clears the bitmap as it scans, so the next collection starts from a clean map. After reset, and after an abandoned collection, the idle engine clears the bitmap in the background, one word per cycle. Free runs are not a moving collector: a live object stays at its address. The first 1024 run headers stay on chip. Further headers go to `PYCORE_GC_RUN_TABLE`. Past that table they occupy the run's first granule, so the list has no fixed cap. Allocator traffic in `S_GC_ALLOC` reads a header and zero-fills the granted line.



Figure 40: Mark-stack entry kinds. An entry is 67 bits: kind, size, and address. `K_TUPLE` is also the continuation kind for 32-byte slots (tuples, list buffers, set tables, dict order). `K_DICTT` continues a dict hash table, whose slots are 64 bytes. `K_OBJ` covers the `OBJECT` header kinds: instance, type, bound method, builtin, bytearray, exception, cell, and function. Strings are traced for the object header; the payload bytes hold no pointers.

Table 6: Set layout, the compact cousin of the dict. No order sidecar. Elements only. The same hash, rich equality, and tombstone rules.

Address	Contents
<code>obj+0</code>	<code>slot_count</code> , <code>used</code>
<code>obj+16</code>	<code>table_ptr</code> (0 when the table is empty)
<code>table + 32i</code>	element value, then element tag

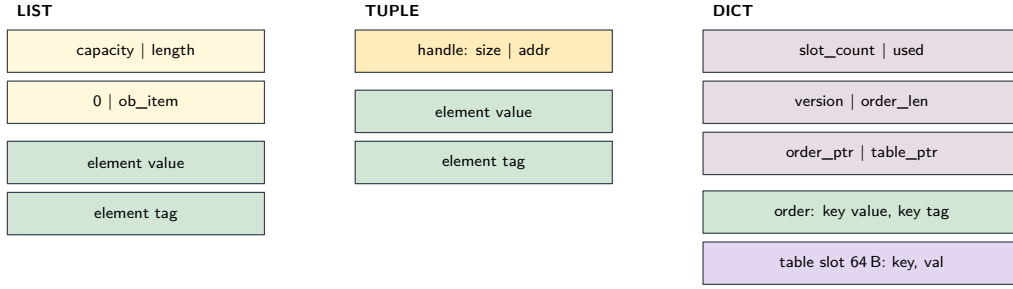


Figure 41: Sequence and dict layout, 16-byte slots. A list is a stable 32-byte object plus a relocatable element buffer. Growing the buffer does not move the handle. Each element is a value slot and a tag slot (32 bytes). An empty list allocates no buffer. A tuple stores its length in the handle, so it has no header. A dict is a stable 48-byte object, an insertion-order key sidecar, and a relocatable open-addressed table. Empty and deleted table slots are UNINIT and TOMBSTONE. New keys append to the sidecar and bump **version**; a value overwrite does neither. Iteration of a dict follows the sidecar, never hash-slot order.

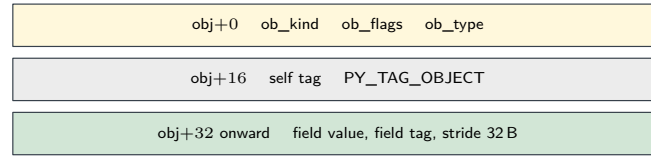


Figure 42: General heap object. The handle is **OBJECT** plus the object address. The kind is not in the tag: **LOAD_ATTR** and **CALL** read **ob_head** first. Kinds in use are instance (dict at field 0), type (dict, base, name), bound method, builtin, exception, cell, and function (code object plus a cell tuple). Lists, dicts, and sets are **MUT_COLLEC**, not **OBJECT**. Strings are **SHORT_STR** or **LONG_STR**. Range has its own tag.

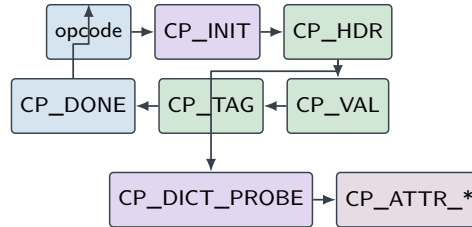


Figure 43: Container microcode (**S_CONTAINER**). Decode sets **dec_is_container** and execute transfers here, skipping **S_MEM** and **S_WB**. The phase register is **container_phase_r**. Shared phases read or write a header, a value slot, and a tag slot, then **CP_DONE** returns to fetch. Dict and set operations insert a probe loop. Attribute operations insert a walk of the instance dict and the type MRO. The arms live in the **pycore_cont_*.svh** files included into the core. Fifty-six of the sixty-four **CONT_*** codes are in use. A recoverable failure sets **trap_marshall_pending_r** and **CP_DONE** leaves for the mailbox instead of fetch.

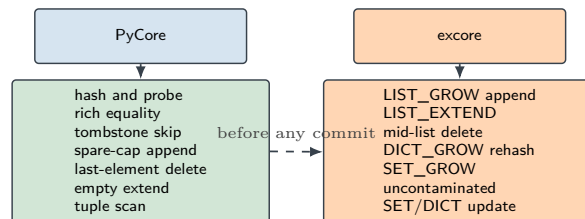


Figure 44: What stays on PyCore, and what is finished by excore. Hashing, equality, and the open-addressed probe stay on the hart. Capacity changes and the long memmoves go to firmware, which must complete the semantic effect rather than resize and retry. The trap is raised before any register-file, heap, or data-memory commit, which is what makes a later **RETRY** legal. No current handler returns **RETRY**; they return **COMPLETED**. Contaminated containers (an **OBJECT** key or element) and tuple-sourced set updates stay on PyCore in **pycore_cont_bulk**.

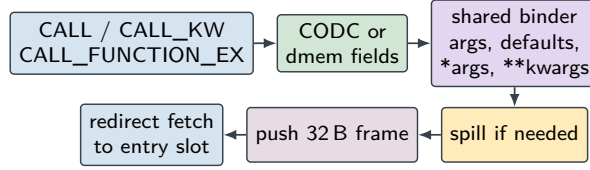


Figure 45: Call, as it sits among the other machines. The phase-by-phase binder is the sibling note `call_fsm.tex`. A user function is a code object, except a closure, which is an `OBK_FUNCTION` wrapping the code object and a cell tuple. `CALL` checks the callable, binds positional and keyword arguments against `co_varnames`, then enters the frame manager. Builtins with a hardware fast path (`len`, `range`, `set`, two-argument `max`, `int` and `str` conversion) dispatch inside this FSM. Other builtin calls raise the recoverable `PY_TRAP_BUILTIN_CALL`. Return pops the descriptor, writes the result into the caller’s ring, and fills any spilled prefix the caller needs.

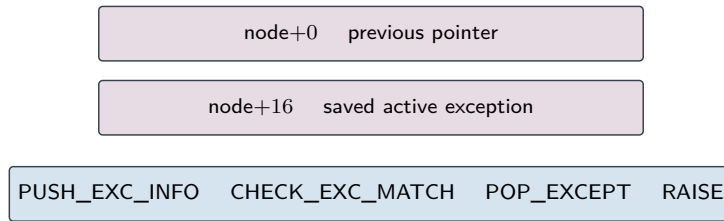


Figure 46: Exception-info stack. The arena is 4 KB at `0xF00000`, 32 bytes per node, depth 128. It is a data-memory LIFO owned by `pycore_exc_stack`, separate from both the operand ring and the call-frame stack. The native-method table and the `StopIteration` sidecar sit in the same arena. `FOR_ITER` exhaustion compares the raised type by identity with the sidecar latched at boot, and redirects to `POP_ITER` without a user handler. A user `raise` with no handler is still fatal `PY_TRAP_RAISE`.

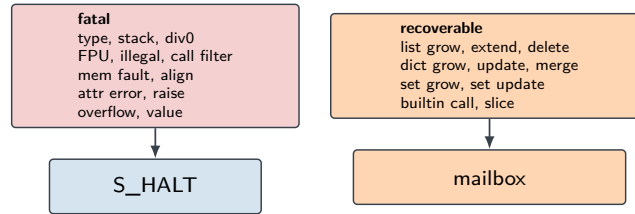


Figure 47: Trap classes. `pycore_trap_recoverable` in `pycore_defs.svh` is the split. With `excore` disabled, a recoverable code is fatal too. `COMPLETED` applies the popped and pushed entries and resumes at the next instruction. `RETRY` redirects fetch to the same PC. `FATAL` enters `pycore_trap` with the firmware’s code. `NEED_HEAP` collects if the collector is enabled, and halts otherwise.

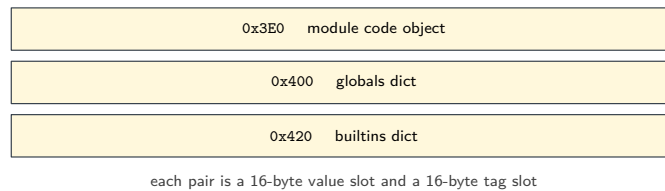


Figure 48: Boot record, read by `S_BOOT` when `BOOT_EN=1`. The walker checks the tags, caches `co_consts` and `co_names`, latches the globals and builtins bases, and redirects fetch to the module entry slot. A serialized code object is eight tagged fields: entry slot, consts, names, packed metadata, defaults, varnames, keyword defaults, and the raw exception table. `LOAD_CONST` indexes `co_consts` with two data reads (value, then tag). `LOAD_GLOBAL` and `LOAD_NAME` probe the frame globals and then the builtins dict; a miss is `MEM_FAULT`.

7.1 Iterator payload

GET_ITER rewrites a list, tuple, range, string, dict, or set into an ITER value. The 128-bit payload is

magic[127:120], kind[119:116], aux[115:96], index[95:64], size[63:32], addr[31:0].

Kinds 0–3 are list, tuple, range, and string. Kind 4 is a heap iterator for a user `__iter__`. Kinds 5 and 6 are dict and set. Dict and set iterators snapshot a mutation counter and raise `PY_TRAP_TYPE` if the container changes size during the loop.

8 Figure index for the paper

A first paper figure set, in the order a systems section usually wants them:

1. Figure 1 — the chip: two harts, one heap.
2. Figure 2 — what is inside PyCore.
3. Figure 5 and Figure 6 — the value the datapath actually carries.
4. Figure 8 and Figure 9 — the register stack.
5. Figure 12 and Figure 13 — memory.
6. Figure 20 — accelerators on one page.
7. Figure 21, Figure 27, Figure 33, Figure 38 — one diagram of how each family works.
8. Figure 41 and Figure 44 — objects, and the cut with excore.

The remaining figures are the working drawings underneath those: cache microarchitecture, the data-port mux, each integer and floating-point datapath, the string engines, the collector’s sweep, the container phases, boot, and the trap split.